Comparative Evaluation of Deep Learning Models for Binary Sentiment Classification on the IMDB Movie Review Dataset

Project Report on

Deep Learning Project [ICT-4442]

Submitted By

ARNAV KUMAR | 230911308

SCHOOL OF COMPUTER ENGINEERING

MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL ACADEMY OF HIGHER EDUCATION

November 2026

EXPERIMENTS INCOMPLETE: this is a progress report, not a completed benchmark.

Abstract

Binary sentiment classification is a foundational task in natural language processing and underpins applications ranging from product-review mining to social-media monitoring. This project compares four deep learning architectures of increasing inductive bias on the IMDB Large Movie Review Dataset (50,000 reviews): a multilayer perceptron over TF-IDF bag-of-words features, a TextCNN over GloVe-300 embeddings, a bidirectional LSTM with an attention layer, and a fine-tuned DistilBERT transformer. All models share one preprocessing pipeline, a stratified 70/15/15 split and a single evaluation harness reporting accuracy, precision, recall, F1, parameter count, training time and inference latency. Test accuracies were: MLP TBD, TextCNN TBD, BiLSTM-Attention TBD and DistilBERT TBD. The study quantifies the accuracy gap between non-transformer models and a pre-trained transformer, and the compute cost at which that gap is bought.

Keywords: sentiment analysis, IMDB, TextCNN, BiLSTM, attention, DistilBERT, transfer learning, model comparison

1. Introduction

Automatically deciding whether a piece of text expresses positive or negative sentiment is one of the most studied problems in NLP. Despite its apparent simplicity, the task exercises most of what makes language hard: sentiment is carried by individual words ("brilliant", "dull"), by short phrases ("not bad at all"), by negation and contrast spanning many words ("I expected to hate it, but..."), and occasionally by sarcasm that only world knowledge resolves.

Each family of neural architecture covered in ICT-4442 encodes a different assumption about which of these signals matter. A bag-of-words MLP assumes word identity is enough. A convolutional network assumes local n-gram patterns matter. A recurrent network assumes word order and long-range context matter. A pre-trained transformer assumes that general language knowledge, learned from large unlabelled corpora, transfers to the target task. Comparing these models under identical conditions shows how much each additional assumption buys, and at what computational cost.

1.1 Objectives

- To evaluate MLP bag-of-words, TextCNN, BiLSTM-Attention and DistilBERT architectures on binary sentiment classification accuracy, F1-score and inference latency on the IMDB dataset under a common evaluation protocol.

- To measure the performance gap between models trained from scratch (MLP, CNN, BiLSTM) and a fine-tuned pre-trained transformer (DistilBERT), and to characterise the cost-benefit trade-off in training time and parameter count.

1.2 Motivation

Sentiment analysis of product and entertainment reviews supports consumer decision making at very large scale. IMDB is the standard benchmark for binary sentiment, so our pooled 70/15/15 split differs from the official split and does not permit direct leaderboard comparisons. The four architectures studied span the practical range of compute budgets, from a model that trains in seconds on a CPU to one that needs a GPU for fine-tuning. For deployment settings such as embedded review analysers or on-device NLP, knowing how close a small model gets to a transformer, and at what fraction of the parameters and latency, is directly useful.

2. Literature Review

- Maas et al. (2011) [1]. Introduced the IMDB Large Movie Review Dataset and learned word vectors that capture both semantic and sentiment information, establishing the benchmark used here.

- Kim (2014) [2]. Showed that a single-layer CNN with multiple filter widths over pre-trained word vectors is competitive with far more complex models on sentence classification, including sentiment. Our TextCNN follows this design.

- Yang et al. (2016) [6]. Hierarchical Attention Networks applied attention over recurrent states to weight the most informative words and sentences in a document, motivating the attention pooling used in our BiLSTM.

- Howard and Ruder (2018) [4]. ULMFiT demonstrated that pre-training a language model and fine-tuning it with discriminative learning rates substantially outperforms training from scratch on sentiment tasks, including IMDB.

- Devlin et al. (2019) [3]. BERT's masked-language-model pre-training and fine-tuning established a general transfer-learning approach for NLP, motivating the transformer baseline. RoBERTa later improved on BERT

through better pre-training choices.

- Sanh et al. (2019) [5]. DistilBERT used knowledge distillation to produce a model 40% smaller and 60% faster than BERT-base while retaining about 97% of its language understanding performance, making it the transformer used in this study.

3. Dataset and Exploratory Analysis

The IMDB Large Movie Review Dataset (Stanford AI Lab) contains 50,000 English movie reviews labelled positive or negative, released as 25,000 training and 25,000 test reviews with 12,500 of each class in each half. There are no missing labels. The raw text version from the HuggingFace datasets library (stanfordnlp/imdb) was used so that a custom preprocessing pipeline could be applied identically to every model.

To obtain a validation set and a single protocol, all 50,000 reviews were pooled and split 70/15/15 with stratification on the label and a fixed seed (42).

Statistic	Value
Reviews (train / val / test)	35,000 / 7,500 / 7,500
Positive fraction	50.0%
Mean review length	~231 words
Other statistics	TBD: run python -m src.eda

Table 1: Dataset statistics after cleaning.

[Figure pending: generated at results\eda\length_distribution.png after running the experiments]

Figure 1: Distribution of review lengths by class; the dashed line marks the 512-token cut-off.

4. Methodology

4.1 Shared preprocessing pipeline

- Lowercasing, removal of HTML line breaks (
) and punctuation; apostrophes are dropped so that negations such as "don't" remain one token ("dont").

- Whitespace tokenisation and a vocabulary of the 10,000 most frequent training tokens, including <pad> and <unk>; the vocabulary is built from the training split only.

- Reviews truncated or padded to 512 tokens. Batches are trimmed to their longest review to avoid wasted computation.

- DistilBERT receives the same cleaned text, tokenised by its own WordPiece tokenizer and capped at 512 sub-word tokens.

4.2 Model 1: MLP over TF-IDF bag-of-words (classical baseline)

Each review is represented by a 10,000-dimensional TF-IDF vector (sub-linear term frequency, fitted on the training split). A feed-forward network with hidden layers of 512 and 128 units, ReLU activations and dropout 0.5 maps this vector to a single logit. Word order is discarded entirely, so this model sets the performance floor achievable from word identity alone.

4.3 Model 2: TextCNN

Following Kim (2014), tokens are embedded with 300-dimensional GloVe vectors (glove.6B, fine-tuned during training; out-of-vocabulary words are initialised randomly). Three parallel 1D convolutions with kernel widths 2, 3 and 4 and 100 filters each act as learned bigram, trigram and four-gram detectors. Max-over-time pooling keeps the strongest response of each filter anywhere in the review; the 300 pooled features pass through dropout 0.5 to a sigmoid output.

4.4 Model 3: BiLSTM with attention

GloVe-300 embeddings feed a two-layer bidirectional LSTM with 128 hidden units per direction. Padded positions are excluded using packed sequences. Instead of using only the final hidden state, an additive attention layer scores each time step, e_t = v^T tanh(W h_t), normalises the scores with a masked softmax and forms a weighted sum of the hidden states. This lets the model focus on the words that determine sentiment and handle negation and contrast that depend on word order.

4.5 Model 4: DistilBERT (fine-tuned)

The pre-trained distilbert-base-uncased encoder (6 transformer layers, 66M parameters) is fine-tuned end to end with a single-logit classification head on the [CLS] representation. Self-attention over the whole review, together with knowledge acquired during pre-training, gives this model the strongest inductive bias of the four.

4.6 Training and evaluation protocol

All models minimise binary cross-entropy on one logit with the AdamW optimiser, gradient clipping at norm 1.0 and fp16 mixed precision on the GPU. After every epoch the validation F1-score is computed; training stops early when it fails to improve, and the best checkpoint is restored before the single evaluation on the test split. The same evaluation harness computes accuracy, precision, recall, F1 and the confusion matrix, counts parameters, records wall-clock training time and measures inference latency both for single reviews and batched over the test set.

Model	LR	Batch	Epoch cap	Patience	Selected
MLP (TF-IDF BoW)	TBD	TBD	TBD	TBD	TBD
TextCNN	TBD	TBD	TBD	TBD	TBD
BiLSTM + Attention	TBD	TBD	TBD	TBD	TBD
DistilBERT	TBD	TBD	TBD	TBD	TBD

Table 2: Selected hyperparameters, read from experiment artifacts.

Two learning rates are compared per architecture using validation F1 only: 1e-3 and 3e-4 for the non-transformers; 2e-5 and 5e-5 for DistilBERT. The selected trial's best validation checkpoint is tested once. This is a limited-budget search, not exhaustive optimisation. The common PyTorch training loop replaces the initially proposed HuggingFace Trainer so all models use the same loss, checkpoint-selection and timing implementation.

5. Experimental Setup

- Hardware: NVIDIA GeForce RTX 4060 Laptop GPU (8 GB).

- Software: Python, PyTorch, HuggingFace Transformers and Datasets, scikit-learn.

- Reproducibility: fixed random seed 42 for the split, initialisation and data order; all hyperparameters in src/config.py.

- Code: run_all.py runs EDA, validation-only tuning, test evaluation, analysis and document generation.

6. Results

Note: some experiments have not been run yet; cells marked TBD are filled in automatically when this report is regenerated after running run_all.py.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Params	Train time (min)	Latency (ms/review)
MLP (TF-IDF BoW)	TBD	TBD	TBD	TBD	TBD	TBD	TBD
TextCNN	TBD	TBD	TBD	TBD	TBD	TBD	TBD
BiLSTM + Attention	TBD	TBD	TBD	TBD	TBD	TBD	TBD
DistilBERT	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 3: Test-set results under the common protocol (7,500 reviews).

[Figure pending: generated at results\tradeoffs.png after running the experiments]

Figure 2: Test accuracy against parameter count, single-review latency and training time (log scales).

[Figure pending: generated at results\learning_curves.png after running the experiments]

Figure 3: Validation loss and F1 per epoch for each model.

7. Discussion

[Key findings are generated from the results once all four models have been run.]

7.2 Effect of inductive bias

The bag-of-words baseline is expected to be strong on IMDB because long reviews contain many sentiment-bearing words, so word identity alone is highly informative. A hypothesis is that errors may involve sentiment reversed by negation or contrast, which TextCNN is designed to capture through short n-grams and the BiLSTM through sequential context. The pre-trained transformer additionally benefits from language knowledge that a model trained on 35,000 reviews cannot acquire on its own.

7.3 Error analysis

For each model the 50 most confidently misclassified test reviews are saved to results/<model>/misclassified.csv. Recurring error categories to examine are: mixed reviews that praise some aspects and criticise others, sarcasm and irony, sentiment expressed only in the final sentences of reviews truncated at 512 tokens, and reviews that discuss the plot's negative events while recommending the film.

These length comparisons and excerpts are descriptive. They do not establish that truncation, sarcasm or negation caused an error. Confidence intervals cover test-sample uncertainty, not variation across training seeds.

7.4 Cost-benefit trade-off

Cost-benefit conclusions are pending measured experiment results.

Training time in the main table is for the selected trial, including validation. Total search training time is separately recorded in comparison.csv. Downloads, preprocessing and final test evaluation are excluded from training time. Latency uses pretokenized inputs and includes data loading and host-to-device transfer; it excludes text preprocessing/tokenization. It is specific to this hardware and batch size.

7.5 Limitations

- Results come from one seed; repeating runs with several seeds would give confidence intervals.

- Punctuation was removed for all models to keep preprocessing identical, which may cost DistilBERT some accuracy compared with feeding it raw text.

- Two learning-rate trials per model are a small search; longer or broader tuning may change rankings.

- GloVe itself is pretrained: only the MLP is entirely trained from scratch. CNN and BiLSTM use pretrained word embeddings.

- The pooled 70/15/15 split matches the synopsis but differs from the official IMDB test benchmark.

- A fixed seed improves reproducibility but does not guarantee bit-identical GPU operations.

- Truncation at 512 tokens discards the end of the longest reviews.

8. Conclusion and Future Work

This project implemented and compared four architectures from different families for binary sentiment classification on IMDB under a single preprocessing pipeline and evaluation harness. The comparison quantifies how much accuracy each additional inductive bias provides and what it costs in parameters, training time and latency, giving practical guidance for choosing a model under a given compute budget.

Future work includes:

- multi-seed runs with confidence intervals and significance tests;

- head-and-tail truncation or hierarchical models for reviews longer than 512 tokens;

- larger pre-trained models (RoBERTa, DeBERTa) and parameter-efficient fine-tuning (LoRA);

- quantisation and distillation of DistilBERT into TextCNN for on-device deployment;

- visualising BiLSTM attention weights to interpret its decisions.

References

[1] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning Word Vectors for Sentiment Analysis," in Proc. ACL, pp. 142-150, 2011.

[2] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in Proc. EMNLP, pp. 1746-1751, 2014.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL, pp. 4171-4186, 2019.

[4] J. Howard and S. Ruder, "Universal Language Model Fine-Tuning for Text Classification," in Proc. ACL, pp. 328-339, 2018.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[6] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical Attention Networks for Document Classification," in Proc. NAACL, pp. 1480-1489, 2016.

[7] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in Proc. EMNLP, pp. 1532-1543, 2014.

Appendix A: Reproducing the Results

pip install torch --index-url https://download.pytorch.org/whl/cu128

pip install -r requirements.txt

python run_all.py

python docs/build_report.py

python docs/build_slides.py